

Tarefa 3

Efeitos

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis

PEDs: Caroline Nakazato, Giovanne Mariano, Gustavo Moraes e
Marcos Freire

PADs: Caio, Lia, Paulo e Vinicius

1. Descrição Geral

Ao longo dos laboratórios anteriores, foi implementado um sistema de combate com um herói, inimigos e gerenciamento de baralho com uso adequado de herança, tornando o código mais fácil de manter. No entanto, o jogo só conta com cartas de dano e defesa, assim como inimigos bem simples, o que o torna desinteressante e pouco estratégico. Na verdade, além destas cartas simples, a profundidade estratégica de Slay the Spire depende fortemente da aplicação de efeitos em entidades, que foram até agora ignorados em nossa implementação.

O objetivo deste laboratório é a implementação de um sistema robusto de efeitos que permitem interações poderosas entre cartas. Esta implementação será feita por meio do padrão de design *Observer*, com o objetivo de exercitar fundamentos de POO, como polimorfismo, composição e encapsulamento.

1.1. Entendendo efeitos



Figura 1: Um inimigo afligido por 3 acúmulos do efeito veneno.

Na Figura 1, podemos ver um inimigo afligido por 3 acúmulos do efeito veneno. Ao final do turno do jogador, o inimigo sofrerá 3 de dano, e consumirá um dos acúmulos de veneno, ficando com 2. Cada efeito em Slay the Spire pode ter uma quantidade de acúmulos, que chamaremos de *X*, e um resultado que depende da quantidade de acúmulos. Vemos abaixo alguns exemplos de efeito:

- **Veneno:** Ao final do turno do jogador, o inimigo sofre *X* de dano, e reduz os acúmulos de veneno em 1.
- **Força:** Aumenta o dano de ataque em *X*.
- **Destreza:** Aumenta o bloqueio recebido em *X*.

Neste laboratório, você deve implementar pelo menos 2 efeitos, que podem ser ou dentre estes 3, ou tirados das páginas de [buffs](#) e [debuffs](#) da wiki de Slay the Spire, ou novos efeitos inventados por você. Em todo caso, todos os efeitos implementados devem ser descritos no README do projeto.

1.2. Aplicando efeitos

Para os nossos objetivos, efeitos podem ser aplicados de duas maneiras:

- **Cartas** podem aplicar efeitos em entidades. As cartas mostradas na Figura 2, por exemplo, estão relacionadas à aplicação do efeito veneno.
- **Inimigos** podem aplicar efeitos no herói, em si, ou em outros inimigos. O inimigo na Figura 1, por exemplo, pretende aplicar um efeito positivo em si no lugar de atacar.

Neste laboratório, devem ser implementadas pelo menos 2 cartas que interagem com efeitos, e inimigos devem ser capazes de aplicar efeitos.

Você pode procurar inspiração para cartas nas listas de cartas das wikis de [Slay the Spire](#) e [Slay the Spire 2](#), que lançou recentemente.



Figura 2: Cartas relacionadas ao efeito veneno em Slay the Spire 2.

2. Objetivos

Ao final deste laboratório, o aluno deverá ser capaz de:

- Compreender e aplicar o padrão de design **Observer** para implementar sistemas baseados em eventos, sem comprometer o princípio da responsabilidade única;
- Modelar e implementar **efeitos acumuláveis** aplicados a entidades em um sistema de combate;
- Estender um sistema existente adicionando novas funcionalidades sem modificar excessivamente o código já implementado;
- Integrar novos comportamentos ao fluxo de combate por meio de cartas, inimigos e efeitos;
- Utilizar controle de versão de forma organizada durante o desenvolvimento, incluindo o uso de **branches**;
- Documentar o funcionamento do projeto em um arquivo **README**.

3. Atividades

Esta seção descreve a implementação do laboratório. O objetivo é implementar efeitos por meio do uso do padrão de design *Observer*. As classes apresentadas abaixo representam a estrutura mínima esperada, mas você é livre para adicionar novos atributos, métodos ou classes auxiliares.

As seguintes diretrizes gerais devem ser observadas:

- **Princípios de POO:** Sempre que possível, mova atributos e métodos comuns para classes abstratas. Quando relevante, aplique os conceitos aprendidos em aula, como encapsulamento e polimorfismo;
- **Liberdade Criativa:** Você pode expandir o conjunto de cartas, criar novos inimigos ou adicionar classes auxiliares. O tema do jogo continua sendo de livre escolha;

- **Autonomia Técnica:** Alguns detalhes de implementação são intencionalmente deixados em aberto. Espera-se que você:
 - Escolha tipos de variáveis adequados para cada atributo;
 - Defina parâmetros apropriados para cada método;
 - Tome decisões de design coerentes com os princípios de orientação a objetos.

Neste laboratório, pelo menos 2 novos efeitos que impactem o fluxo do jogo devem ser implementados, bem como pelo menos 2 cartas que aplicam efeitos. Inimigos, também, devem ser capazes de aplicar efeitos. Por exemplo, você pode implementar os efeitos veneno e força descritos na Seção 1.1, implementar 2 cartas que utilizam o efeito veneno, e dar a um inimigo a capacidade de aplicar força em si mesmo.

3.1. O padrão de design *Observer*

Efeitos em Slay the Spire são aplicados em momentos diferentes do combate. Por exemplo, veneno é aplicado ao final do turno do jogador, enquanto que força é aplicada sempre que uma entidade afligida ataca. Surge, portanto, a questão de como aplicar o efeito certo no momento certo. Uma solução ingênua seria, ao final do turno do jogador, verificar se cada entidade está envenenada e aplicar o dano correspondente a ela. No entanto, isto fere o princípio de encapsulamento, e deixa a classe que lida com a lógica de turno responsável por implementar todos os efeitos do jogo, o que torna a manutenção difícil.

Uma solução mais elegante pode ser desenvolvida por meio do uso do padrão de design *Observer*, no qual uma série de *Subscribers* podem se inscrever em um *Publisher* de modo que eles sejam notificados sempre que acontecer um evento específico no *Publisher*. No nosso caso, aplicações do efeito de veneno em entidades podem se inscrever em um método na classe responsável pelo fluxo do jogo que as notifica sempre que o turno do jogador acabar para que elas possam aplicar o dano. Neste caso, a classe **Efeito** age como *Subscriber*, e a classe responsável pelo fluxo do jogo age como *Publisher*.

3.1.1. Classe *Publisher*

- **Atributos mínimos**
 - `subscribers` — Lista de *Subscribers*.
- **Métodos mínimos**
 - `inscrever` — Inscreve um *Subscriber* para ser notificado;
 - `desinscrever` — Desinscreve um *Subscriber* de modo que ele não seja mais notificado;
 - `notificar` — Chama um método predefinido em todos os *Subscribers*.

3.1.2. Classe *Subscriber*

- **Métodos mínimos**
 - `serNotificado` — Chamado pelo método `notificar` do *Publisher*.

3.2. Classes

Estas são as classes mínimas que devem ser implementadas. Algumas destas classes são abstratas e devem ser herdadas por classes implementadas para cumprir os outros requisitos do laboratório. Esta lista também inclui modificações que devem ser feitas em classes já implementadas.

3.2.1. Classe Abstrata *Efeito*

Representa efeitos, positivos e negativos, que podem ser aplicados em entidades. Deve agir como *Subscriber*.

- **Atributos mínimos**
 - `nome` — Nome do efeito;
 - `dono` — Entidade possuidora deste efeito;
 - `acumulos` — Total de acúmulos de um efeito. O significado da quantidade de acúmulos depende do efeito.
- **Métodos mínimos**
 - `getString` — Retorna uma *String* representando o efeito, que inclui o nome do efeito e o total de acúmulos.

3.2.2. Modificação na Classe Abstrata Entidade

Implementada no Laboratório 2. Representa qualquer personagem presente na batalha, como heróis e inimigos.

- **Atributos mínimos**

- **efeitos** — Lista de efeitos aplicados nesta entidade.

- **Métodos mínimos**

- **aplicarEfeito** — Aplica um efeito com X acúmulos. Se o jogador não tem este efeito, uma nova instância de Efeito é criada. Caso contrário, soma a quantidade de acúmulos no efeito já existente.

3.3. Implementações mínimas

Além disso, devem ser implementados:

- Pelo menos 2 efeitos que mudam o fluxo do jogo. Use a Seção 1.1 como inspiração.
- Pelo menos 2 cartas que aplicam efeitos. Use a Seção 1.2 como inspiração.
- Inimigos capazes de aplicar efeitos além de atacar. Inimigos devem ter uma série de ações possíveis e selecionar entre eles a cada turno (por exemplo, de maneira rotativa ou aleatória).

Os efeitos implementados devem ser impressos no terminal junto com as informações de combate. Todos os efeitos implementados devem ser descritos no README do projeto.

3.4. Implementações sugeridas

Embora não seja estritamente necessário para completar a tarefa, recomendamos fortemente a implementação dos itens a seguir para simplificar a lógica do programa e organizar melhor o fluxo do combate. Essas sugestões não definem uma única forma correta de implementação, mas indicam decisões de design que costumam levar a soluções mais claras e fáceis de manter.

3.4.1. Classe responsável pelo fluxo do combate

Uma abordagem recomendada é criar uma classe dedicada a gerenciar o fluxo da batalha, por exemplo chamada **Combate** ou **GameManager**. Essa classe pode ser responsável por:

- Controlar a ordem dos turnos do jogador e dos inimigos;
- Executar ações como ataques e uso de cartas;
- Notificar os efeitos quando eventos relevantes do combate ocorrerem.

Nessa abordagem, essa classe atua como o **Publisher** do padrão *Observer*. Ela mantém uma lista de **Subscribers** (efeitos ativos) e notifica esses efeitos sempre que ocorre um evento relevante no combate.

Centralizar a lógica de turno e os eventos do jogo em uma única classe geralmente torna o código mais organizado e evita que múltiplas classes precisem conhecer os detalhes do fluxo de combate.

3.4.2. Eventos do combate

Outra decisão importante é definir **quais eventos do combate podem ser observados pelos efeitos**. Diferentes efeitos podem reagir a diferentes momentos do jogo. Alguns exemplos de eventos que podem ser úteis incluem:

- fim do turno do jogador;
- início ou fim do turno de uma entidade;
- momento em que uma entidade realiza um ataque;
- momento em que uma entidade recebe dano.

Uma forma simples de organizar isso é definir um conjunto de tipos de evento (por exemplo, utilizando uma enumeração ou constantes) e fazer com que o método de notificação informe qual evento ocorreu. Cada efeito então decide se deve reagir ou não àquele evento.

3.4.3. Inscrição e remoção de efeitos

Quando um efeito é aplicado a uma entidade, ele deve ser adicionado à lista de efeitos dessa entidade. Além disso, como os efeitos precisam ser notificados sobre eventos do combate, é importante garantir que eles também sejam registrados no **Publisher** responsável pelo combate.

Da mesma forma, quando um efeito deixa de existir (por exemplo, se seus acúmulos chegam a zero), ele deve ser removido tanto da entidade quanto da lista de **Subscribers** do **Publisher**.

Organizar esse processo de inscrição e remoção de forma clara ajuda a evitar efeitos “fantasma” que continuam recebendo notificações mesmo após terem sido removidos do jogo.

3.4.4. Acesso a informações do combate

Alguns efeitos podem precisar acessar informações adicionais sobre o estado do combate para funcionar corretamente. Por exemplo:

- saber qual entidade está atacando;
- saber qual entidade é o alvo de um ataque;
- acessar o estado atual da batalha.

Por esse motivo, pode ser útil permitir que o método de notificação receba alguma forma de acesso ao estado atual do combate (por exemplo, uma referência ao objeto responsável pelo combate). Dessa forma, os efeitos conseguem reagir de maneira mais flexível aos eventos do jogo.

Essas decisões de design não são obrigatórias, mas costumam resultar em uma implementação mais modular e alinhada com os princípios de orientação a objetos.

3.5. Pontos de Atenção

Antes de finalizar o laboratório, verifique se:

- Os efeitos implementados herdam corretamente da classe abstrata **Efeito**;
- Os efeitos foram implementados por meio do uso do padrão de design *Observer*;
- Cartas e inimigos podem aplicar efeitos, e inimigos escolhem entre múltiplas ações possíveis a cada turno;
- O funcionamento dos efeitos fica claro pelo que é impresso no terminal;
- Os efeitos implementados foram descritos claramente no README.

3.6. Desafio Extra (+0,5 ponto extra)



Figura 3: Múltiplos inimigos em um combate.

Até o momento, implementamos combate contra um único inimigo. No entanto, uma das fontes de complexidade em Slay the Spire é lidar com múltiplos inimigos ao mesmo tempo, cada um com uma intenção distinta. Um exemplo de combate contra múltiplos inimigos ao mesmo tempo é mostrado na Figura 3.

Como desafio extra neste laboratório, você deve modificar seu código para permitir que o jogador enfrente múltiplos inimigos ao mesmo tempo, de modo que:

- Cada inimigo tenha vida e ações independentes;
- Quando uma carta tiver um alvo único, o jogador deve escolher que inimigo alvejar;

- Ao rodar o código, o jogador é colocado em um combate com mais de um inimigo. Você pode implementar isto com múltiplas instâncias do mesmo inimigo ou com múltiplos inimigos distintos.

Este desafio é inteiramente opcional, mas concluí-lo corretamente vale até +0,5 pontos extra na nota deste laboratório.

4. Avaliação

A nota do laboratório será composta pelos seguintes critérios:

1. Implementação do Padrão Observer (3,0 pontos)

- Implementação correta de `Publisher` e `Subscriber`;
- A classe `Efeito` deve agir `Subscriber`;
- Efeitos devem ser registrados e removidos corretamente do `Publisher`;
- O sistema de notificações deve ser utilizado para reagir a eventos do combate.

2. Sistema de Efeitos e Integração com o Combate (3,0 pontos)

- Implementação de pelo menos dois efeitos que modificam o fluxo do jogo;
- Implementação de pelo menos duas cartas capazes de aplicar efeitos;
- Inimigos capazes de aplicar efeitos além de causar dano;
- Efeitos acumuláveis funcionando corretamente;
- Reação correta dos efeitos aos eventos do combate.

3. Organização do Projeto e Qualidade do Código (2,0 pontos)

- Cada classe está em seu próprio arquivo `.java`;
- Código legível, bem formatado e com nomes adequados;
- Uso consistente de encapsulamento;
- Comentários quando necessários para trechos não triviais.

4. Documentação e Uso de Controle de Versão (2,0 pontos)

- O repositório contém um arquivo `README.md` que descreve claramente:
 - o funcionamento do jogo até o momento (incluindo os efeitos implementados);
 - como compilar o projeto;
 - como executar o programa.
- O histórico do repositório demonstra uso adequado de controle de versão, incluindo:
 - uso de **branches** para desenvolvimento de novas funcionalidades ou refatorações;
 - mensagens de commit claras e descritivas;
 - integração das alterações ao ramo principal do projeto.

5. Desafio extra (até +0,5 ponto extra)

- Conclusão correta do desafio proposto na Seção 3.6.

6. Penalidade por Não Compilação

Caso o projeto:

- Não compile utilizando Java 25; ou
- Não execute corretamente no ambiente de correção;

será aplicada uma penalidade de **-5,0 pontos** sobre a nota obtida.

A nota final mínima é **0,0** (não haverá notas negativas).

5. Entrega

A submissão é realizada exclusivamente via repositório no **GitHub**. Para isso:

- Gere uma **release (tag)** com a identificação do laboratório, seguindo o formato: **tarefaX-RA1-RA2**;
- Gere uma release no GitHub a partir dessa tag;
- Envie o link da release no Google Classroom.

Caso optem por manter o repositório **privado**, adicionem os PEDs da disciplina como colaboradores:

- Caroline Nakazato – @CarolineNakazato
- Giovanne Mariano – @giolucasd
- Gustavo Moraes – @moraguma
- Marcos Freire – @marcfreir

Lembrem-se que:

- O prazo de submissão é até o dia **26/03/2026 às 23h59**.
- Entregas atrasadas serão penalizadas em **25 % da nota obtida**.
- Os horários de laboratório e atendimento podem (e devem) ser utilizados para tirar dúvidas relacionadas tanto à submissão quanto ao desenvolvimento do laboratório.

5.1. Compilação e Execução

Durante a avaliação, os PEDs utilizarão os seguintes comandos, a partir da root do repositório, para compilar e executar o projeto:

```
javac -d bin $(find src -name "*.java")
java -cp bin App
```

Certifique-se de que existe uma classe chamada **App** contendo o método:

```
public static void main(String[] args)
```

Essa classe será utilizada como ponto de entrada para a execução do programa.
Dessa forma, a compilação e a execução ocorrerão corretamente no ambiente de avaliação.